AI-Based Smart Timetable Generator Using Constraint Optimization

Purpose: Ensure continuous collaboration, integration, and visible contribution from both team members.

Working Model:

You build foundation → Teammate builds logic
→ You integrate → Teammate improves
→ Repeat

This ensures:

- Continuous collaboration
- No isolated work
- Easy faculty updates
- Early integration
- No last-minute merge issues

Responsibility Split

Your Role (System Development)

Frontend + Backend + Database + API + Integration

You handle:

- Flask backend
- MySQL database
- HTML/CSS/JS frontend
- CRUD operations
- Admin dashboard
- API layer
- Export system (PDF/Excel)
- Final UI integration

Teammate Role (AI & Optimization)

Scheduling Engine + Constraint Optimization

He handles:

- OR-Tools
- Constraint modeling
- Clash detection

- Timetable generation
- Optimization logic
- Load balancing
- Smart scheduling

ALTERNATING DEVELOPMENT ROADMAP

Step	Your Work	Teammate Work	Integration Output
1	Setup Flask, MySQL, folder structure	Research OR-Tools + timetable optimization flow	Common architecture finalized
2	Build Departments CRUD	Define required timetable constraints	Constraint list shared with you
3	Build Faculty CRUD (max_hours, department mapping)	Design hard constraints logic	Faculty-related constraints documented
4	Create faculty_subject_mapping table	Build faculty clash logic prototype	Mapping schema finalized
5	Build Subjects CRUD (lab/theory, weekly hours)	Build subject allocation logic	Subject rules integrated
6	Update DB based on his requirements	Validate subject-hour allocation logic	Subject-hour completion ready
7	Build Rooms/Labs CRUD	Build room clash prevention	Room allocation compatible
8	Add room type + capacity support	Add lab-room-only constraint	Lab scheduling logic aligned
9	Build Sections/Class CRUD	Build class overlap prevention	Class conflict prevention ready
10	Build Working Days + Time Slots module	Create empty timetable grid logic	Scheduling framework ready
11	Create API /optimizer-input	Consume API data in OR-Tools	First real integration
12	Test API with dummy data	Generate basic timetable prototype	Prototype timetable generated
13	Create timetable database table	Add faculty availability constraint	Faculty availability supported
14	Add faculty preference UI	Add preferred timing soft constraint	Preference scheduling ready
15	Add lunch break settings	Implement mandatory lunch constraint	Lunch conflict prevention

Step	Your Work	Teammate Work	Integration Output
16	Add workload fields + validations	Implement workload balancing	Smart faculty load allocation
17	Add consecutive lab settings	Implement consecutive lab scheduling	Lab continuity working
18	Update APIs for advanced constraints	Add multi-department optimization	Cross-department scheduling ready
19	Build timetable display UI	Improve optimization quality	Better generated schedules
20	Add filters (faculty/section/room wise)	Add heavy subject distribution logic	Smarter timetable balancing
21	Create /generate-timetable integration route	Return optimized timetable JSON	Full backend ↔ AI integration
22	Debug frontend rendering	Fix conflicts/edge cases	Stable generation flow
23	Build export to PDF	Validate export compatibility	PDF export ready
24	Build export to Excel	Verify timetable formatting	Excel export ready
25	Add admin dashboard analytics	Add optimization suggestions	AI recommendation support
26	Full system testing	Constraint testing	Conflict-free timetable testing
27	UI polishing + validation fixes	Optimization tuning	Better performance
28	Final integration testing	Final optimizer tuning	Stable MVP ready
29	Documentation + screenshots	Algorithm documentation	Final report ready
30	Viva preparation + demo	Viva preparation + optimization explanation	Final submission ready

---

HARD CONSTRAINT OWNERSHIP

Constraint	You (DB/UI/API)	Teammate (Logic)
No faculty clash	Faculty data + API	Clash prevention logic
No room clash	Room DB	Room allocation logic
No class overlap	Section DB	Class scheduling logic
Faculty-subject mapping	Mapping table	Mapping validation
Department mapping	Department DB	Department-aware scheduling
Weekly hours completion	Subject hours field	Hour completion logic
Lab-only room allocation	Room type	Constraint implementation
Consecutive lab scheduling	Lab settings field	Consecutive allocation logic

Constraint	You (DB/UI/API)	Teammate (Logic)
Faculty availability	Availability UI	Availability constraints
Lunch break	Lunch config	Break enforcement
Room capacity	Capacity DB	Capacity validation
Working day/period limits	Timeslots DB	Scheduling limit logic
Maximum workload	max_hours field	Workload balancing
Fixed academic schedule	Schedule settings	Constraint locking
No double booking	Timetable table	Conflict prevention

Collaboration Rule (MANDATORY)

After every completed step:

You Must Share

- Database changes
- API changes
- New fields/tables
- Frontend updates

Teammate Must Share

- Constraint logic added
- Inputs required
- Outputs expected
- Missing data fields
- OR-Tools changes

Integration Checkpoints

Checkpoint 1 (After Step 10)

Goal:

All academic data available

Must be ready:

- departments
- faculty
- subjects
- rooms

- sections
- time slots

Checkpoint 2 (After Step 12)

Goal:

First timetable generated

Even if imperfect.

Checkpoint 3 (After Step 20)

Goal:

Constraint optimization working

Hard constraints should pass.

Checkpoint 4 (After Step 30)

Goal:

Submission-ready project

Includes:

- AI optimization
- CRUD system
- Admin dashboard
- PDF/Excel export
- Testing
- Documentation

Weekly Progress Update Format

Every 3–4 days:

Completed:
Current blocker:

Database/API changes:
Constraint updates: